

Performance Testing

Date	1 July 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual testing (stopwatch-based timing); no automated load-testing tool (JMeter/Locust) used, as this is a single-user AI inference application, not a multi-user API service
Type of Testing	Functional Testing, Response Time Testing
Target Module / API	Core evaluation pipeline — Whisper transcription, Sentence-BERT similarity scoring, Librosa audio analysis, Gemini API feedback generation
Test Environment	Hosted (Hugging Face Spaces — Docker, CPU Basic, free tier)
Test Date	29.06.2026

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Upload and analyse a 52-second audio clip (Machine Learning concept)	1	52	Accurate transcript, similarity score, and feedback generated
2	Upload and analyse audio for a different concept (Cloud Computing)	1	~50	Correct reference concept loaded and matched dynamically

3	Trigger Gemini API rapidly (back-to-back requests)	1	<10	Graceful retry/fallback instead of application crash
4	Access app after Hugging Face Space wakes from idle sleep (cold start)	1	~30 (cold start overhead)	App loads successfully and completes analysis despite added startup delay



Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	~15-18 seconds	Pass	Higher due to AI model inference (Whisper + Sentence-BERT + Gemini API call); not a lightweight API, so standard 2-sec web targets don't directly apply
2	Response Time (Max)	< 5 seconds	~25-30 seconds (cold start)	Pass	Cold-start delay when Space wakes from sleep; reasonable for an ML-based analysis tool
3	Throughput (Req/sec)	—	~0.05 req/sec (1 request every ~18-20 sec)	N/A	Single-user, sequential AI pipeline; not designed for high-throughput concurrent requests
4	Error Rate	< 1%	0% functional errors after fixes; 1 transient 429 rate-limit error (resolved via retry logic)	Pass	No crashes after ffmpeg and rate-limit fixes were applied
5	CPU Utilization	< 80%	Not directly measurable on free-tier Spaces (no resource dashboard access)	N/A	Free tier doesn't expose detailed resource metrics to the user
6	Memory Utilization	< 80%	Not directly measurable on free-tier Spaces (no resource dashboard access)	N/A	Same limitation as above

Step 4: Observations & Analysis

Key Findings:

The application functions correctly and produces accurate results, but response times are significantly higher than typical lightweight web-service targets because each request involves real AI model inference (speech transcription, semantic embedding comparison, and an external LLM API call) rather than a simple database or API lookup. This is expected and acceptable for an AI-based evaluation tool, where users expect a short processing wait in exchange for AI-powered analysis.

Bottlenecks Identified:

1. Cold-start delay when the Hugging Face Space wakes from inactivity sleep.
2. Gemini API free-tier rate limit (5 requests/minute) caused one transient 429 error during rapid testing.
3. Initial deployment failure due to missing ffmpeg dependency in the Docker container, which Whisper requires for audio decoding.

Optimization steps taken:

1. Added ffmpeg to the Dockerfile's system dependencies, resolving the audio-loading failure.
2. Implemented automatic retry logic (up to 3 attempts with delay) in the feedback generation module to gracefully handle Gemini API rate limits.
3. Fixed file path resolution in the core pipeline to use absolute paths, ensuring reliable file loading regardless of server working directory.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



